IA & CYBERSÉCURITÉ — ITÉRATION 4

Évaluer un modèle de détection SSH

Olivier HIMBLOT • 22 octobre 2026 • Dossier pour le formateur

Compétence : **Évaluer les performances et les limites d'un modèle de détection d'anomalies.**

Critère : mesure sur des données nouvelles au-delà du taux global ; explication des erreurs, biais, dérives et vulnérabilités ; définition des conditions d'exploitation.

Attendu	Preuve et état
Données nouvelles et métriques	Septembre entraîne, octobre évalue ; matrices, précision/rappel et charge documentés.
Erreurs, biais et dérives	Profils de FP/FN analysés ; exclusions, apprentissage court, étiquettes et évolution du SI explicités.
Vulnérabilités	Simulations4.6 exécutées : extraction locale, fractionnement, liste blanche et empoisonnement ; kit et V2 distingués.
Conditions d'exploitation	Conditions4.8 rédigées ; pilote4semaines et critères proposés, sans approbation RSSI/SOC ni déploiement revendiqués.
Exécution et reproductibilité	Deux captures VM ; tests de répétabilité et contrôles dans un environnement distinct de l'assistant.

État de la preuve

Données fictives du scénario Hexalp. Rapport rédigé avec assistance à partir des sorties fournies et vérifiées. Les résultats, essais de vulnérabilité et conditions proposées sont documentés. Leur portée reste celle du TP ; la validation opérationnelle des conditions n'est pas établie. Ce dossier ne prétend ni à un déploiement réel ni à une validation globale du modèle.

Fichiers de preuve associés

Dans IA_J4_ressources_apprenant/ia01-j4 : detection_ssh_v2.py ; requirements.txt ; alertes_octobre.csv (239 lignes de données) ; rapport_evaluation.md ; preuves/execution_v2.txt ; preuves/verification_v2.json. Le PDF présente les résultats ; conserver le script et le CSV comme fichiers exploitables joints à la remise.

Organisation : rapport complet sur les trois pages suivantes ; annexes séparées pour la chaîne de décision et les captures sélectionnées.

Rapport d’évaluation — Détection SSH Hexalp

Olivier HIMBLOT — 22 octobre 2026 — Itération 4, version complète

Données fictives. Rédaction et vérifications locales avec assistance ; conditions d’exploitation proposées, non déployées.

Résumé pour Nadia

Pas de mise en service opérationnelle générale en l’état ; poursuite d’un pilote local supervisé pendant quatre semaines.

Le modèle reste une aide, avec fail 2ban et les contrôles existants ; aucune action de blocage ne lui est déléguée.

Le pilote exige des données contrôlées, des versions protégées et des alertes réellement qualifiées, surplus compris.

Avant élargissement, Nadia et le SOC doivent valider la capacité, les erreurs acceptables et une évaluation sur une période future intacte.

Réévaluation hebdomadaire, bilan formel au plus tard à quatre semaines, et immédiate après incident manqué ou changement significatif.

Données et préparation

Une ligne = une IP pendant une heure. **Septembre (14–27/09)** : 1 704 lignes, 117 attaques (6,87 %) ; après exclusion des 90 volumes absents, **1 614 lignes et 108 attaques**. **Octobre (28/09–11/10)** : 2 119 lignes, 215 attaques (10,15 %) ; après préparation, **2 105 lignes et 211 attaques**.

Changements : nouvelle sonde en Ko le 29/09, proxy Zabbix le 30/09, plage VPN le 01/10, panne NetFlow le 04/10, scan le 06/10. Le journal DSI aide à qualifier sans légitimer automatiquement chaque activité.

La V2 complète les Mo absents par **Ko/1 024**, puis conserve le choix J3 : suppression des vrais volumes manquants. Les **14 exclusions d’octobre, dont quatre attaques**, restent hors couverture. Origine codée externe 0/1. Ordre des entrées : heure, externe, échecs, succès, comptes, durée, volume. IP/date restent pour l’analyse ; fail 2ban et étiquette exclus des entrées. Contrôles de schéma, valeurs, périodes et manques ; arrêt au-delà de **10 %**, seuil pédagogique non approuvé. Un contrôle réussi ne garantit pas toute la qualité du lot.

Protocole et résultats

Graine **42**, profondeur **3**, contamination **0,03**. L'arbre apprend sur 1 129 lignes de septembre, contrôle sur 485 ; IF sur tout septembre préparé, sans étiquette. Aucun ajustement sur octobre ; les étiquettes ne servent qu'au bilan. Le script J3 simplement relancé sur octobre réentraîne et teste dans ce mois : le « 99 % » ne mesure pas le transfert temporel. Une accuracy élevée masque aussi déséquilibre des classes et exclusions.

Octobre, 2 105 lignes	VN	FP	FN	VP	Précision	Rappel	Alertes S40 / S41
Arbre	1 868	26	17	194	88,18 %	91,94 %	93 / 127
Isolation Forest	1 883	11	175	36	76,60 %	17,06 %	29 / 18
Union	1 857	37	9	202	84,52 %	95,73 %	104 / 135
fail 2ban historique	1 894	0	150	61	100 %	28,91 %	61 au total

VN/FP/FN/VP = normales correctement ignorées / fausses alertes / attaques manquées / attaques repérées. Septembre : arbre testé **453 VN,0 FP,4 FN,28 VP**, précision 100 %, rappel 87,5 %. Le rappel octobre augmente ; la précision baisse. Sur septembre connu, IF affichait 89,80 %/40,74 % et union 95,50 %/98,15 % (précision/rappel), mesures incluant l'entraînement, comparaison descriptive seulement. Union :45/66 puis 104/135 alertes/semaine.

L'union repère 142 attaques non signalées par fail 2ban, mais manque une des 61 que celui-ci signale (60 communes). **Conserver les deux moyens.** L'exfiltration du 09/10 à 3 h est retrouvée par IF après conversion :8 898 637 Ko =8 690,075 Mo ; l'arbre l'ignore.

Erreurs et coût

Les 37 FP : **proxy Zabbix 26** (quatre comptes, branche de l'arbre), **sauvegarde 10**, **scanner 1** (IF). À10 min :4 h 20 +1 h 40 +10 min = **6 h 10** théoriques de fausses alertes. Les 239 alertes demanderaient 39 h 50 ; capacité 60 sur deux semaines, **179 en excédent**. Leur ordre réel de lecture est inconnu ; score faible ne signifie pas gravité élevée. Pas de liste blanche globale.

Les **neuf FN** : quatre transferts VPN de 6 629–7 617 Mo, trois activités internes sur trois comptes, deux succès externes sur un compte. Durées/comptes paraissent ordinaires à l'arbre, IF ne franchit pas son seuil. Étiquettes de référence, scénarios complets non fournis. Ces neuf FN n'incluent pas les quatre attaques exclues faute de volume.

Limites et pistes

Apprentissage de deux semaines peu représentatif des rôles nouveaux ; annotations de Hugo à relire en cas d'ambiguïté ; agrégation horaire sans identité/destination ni cumul long ; dérive et pertes de collecte ; octobre déjà exploré. Un meilleur score sur ce mois après réglage ne serait pas un nouveau test indépendant.

Ajouter contexte VPN/rôle/destination et fenêtres glissantes, revoir corpus/étiquettes. Piste notebook 02 : réentraîner sur septembre + première semaine d'octobre, tester sur la seconde (**non exécuté**, semaine déjà connue) ; confirmer ensuite sur une période future intacte.

Résistance et inférence

Simulations 4.6 locales, hors système réel : kit médiane/IF 0,05 distinct de V2 exclusion/0,03. Dix sondes révèlent deux frontières locales du kit. Son spraying passe, mais le même plan est repéré par la V2 aux 24 h testées. Fractionner 5 000 Mo en cinq heures passe dans les deux. Une exemption d'IP masquerait 9 000 Mo pourtant signalés.

Parmi les tailles testées, **quatre fausses lignes** suffisent à faire passer le profil ciblé dans la V2 ; avec vingt, il passe malgré une matrice test inchangée. Après 80 observations nocturnes ajoutées à l'entraînement IF, neuf VPN passent, contre quatre initialement. Effets non monotones, aucun seuil universel. Ces essais supposent un futur corpus altéré, pas une modification spontanée du modèle figé.

Autoencodeur 4.7 : SHA-256 conforme au PDF avant/après, fichier inchangé, inférence seule. Sur 2 105 lignes : **6 FP,141 FN,70 VP**, précision 92,11 %, rappel 33,18 %, neuf VPN repérés ; insuffisant pour remplacer l'union. Résultats pédagogiques sur 2 119 lignes séparés. Les captures corroborent les exercices ; les essais complémentaires restent attribués à l'environnement local de vérification.

Conditions d’exploitation

Rubrique	Conditions proposées — validation RSSI/SOC requise
Données	Avant chaque lot : schéma/unités/valeurs, périodes, doublons, manques et journal DSI. Refus si contrat invalide, incident de collecte et exclusions visibles ; seuil 10 % à arbitrer.
Réévaluation	Revue hebdomadaire, bilan à 4 semaines ; immédiate après changement/incident. Candidat sur données futures intactes et étiquettes revues ; cibles proposées rappel≥95 %, précision≥85 %, scénarios critiques sans régression, charge qualifiable. La V2 ne satisfait pas charge/précision ; aucun remplacement automatique.
Protection	Écritures réservées aux mainteneurs/corpus habilités ; revue des ajouts et étiquettes, versions/empreintes confrontées à référence approuvée, retour arrière ; exceptions contextuelles validées, jamais IP entière sans contrôle.
Alertes	Capacité 30/semaine sans suppression du surplus ; priorité métier puis score, stock/âge et escalade. Analyste reçoit mesures/avis/déclencheurs et contexte ; ajouter identifiant, versions, couverture et liens aux sources (pas tous présents dans le CSV).
Interdits	Modèle : aucun ban, désactivation, commande ou changement de configuration ; ni mise à jour ni exemption autonomes. fail 2ban reste indépendant.
Indicateurs	Couverture, alertes/stock/âge, temps, FP/FN et précision/rappel par profil/gravité ; rappel non mesurable tant que FN inconnus.
Responsabilités	Sofia exploite ; administrateurs maintiennent sources/fail 2ban ; Hugo/SOC qualifient/annotent ; mainteneurs préparent les mises à jour ; Nadia décide charge/risque et validation. Rôles proposés, non habilitations établies.

Preuves : V2, requirements,239 alertes, notebook 03/04 exécutés localement, JSON/journaux sous preuves/, captures de l’apprenant. Deux relances V2 identiques vérifiées localement ; répétabilité et identité des fichiers VM par empreinte non démontrées. SHA-256 aussi contrôlé dans la VM. Aucun déploiement ni accord de mise en service établi. Référence : kit J4,4.1–4.8 ; détails et cinq questions à l’éditeur dans le carnet/fiche 4.8.

Annexe — De la collecte à la décision

Chaîne cible proposée en4.8 ; la V2 existante produit un CSV. Les raccordements SOC, enrichissements supplémentaires et procédures ci-dessous ne sont pas démontrés comme déployés.

Maillon / pilote proposé	Fonction et comportement en cas de panne
Journaux — administrateurs/supervision	Collecte automatique. Si sonde/journaux absents : incident de collecte, couverture dégradée ; jamais normal par défaut.
Qualité — Sofia	Schéma, unités, périodes, doublons/manques ; revue du journal DSI. Lot modèle refusé si contrat invalide, traces conservées.
fail2ban — administrateurs bastion	Branche indépendante : règle existante et bannissement déjà autorisé. Signaler toute panne ; le modèle ne le remplace pas.
Préparation et modèles — supervision	Version validée, conversion et exclusion tracées, prédiction. Arrêter si échec ; retour arrière seulement compatible avec les données.
Enrichissement et file — supervision/SOC	Mesures, avis, score, déclencheur ; ajouter version, identité/destination et contexte. Conserver le surplus, suivre âge et priorités ; pas de troncature à30.
Qualification — Hugo/SOC	Analyse humaine, référence aux sources et décision tracée. Indisponibilité : escalade/renfort, pas de validation automatique.
Décision — analyste habilité/RSSI	Réponse selon la procédure ; mise à jour du modèle approuvée et réversible. Aucun ban ou changement de configuration décidé par le modèle.

Cinq questions à l'éditeur

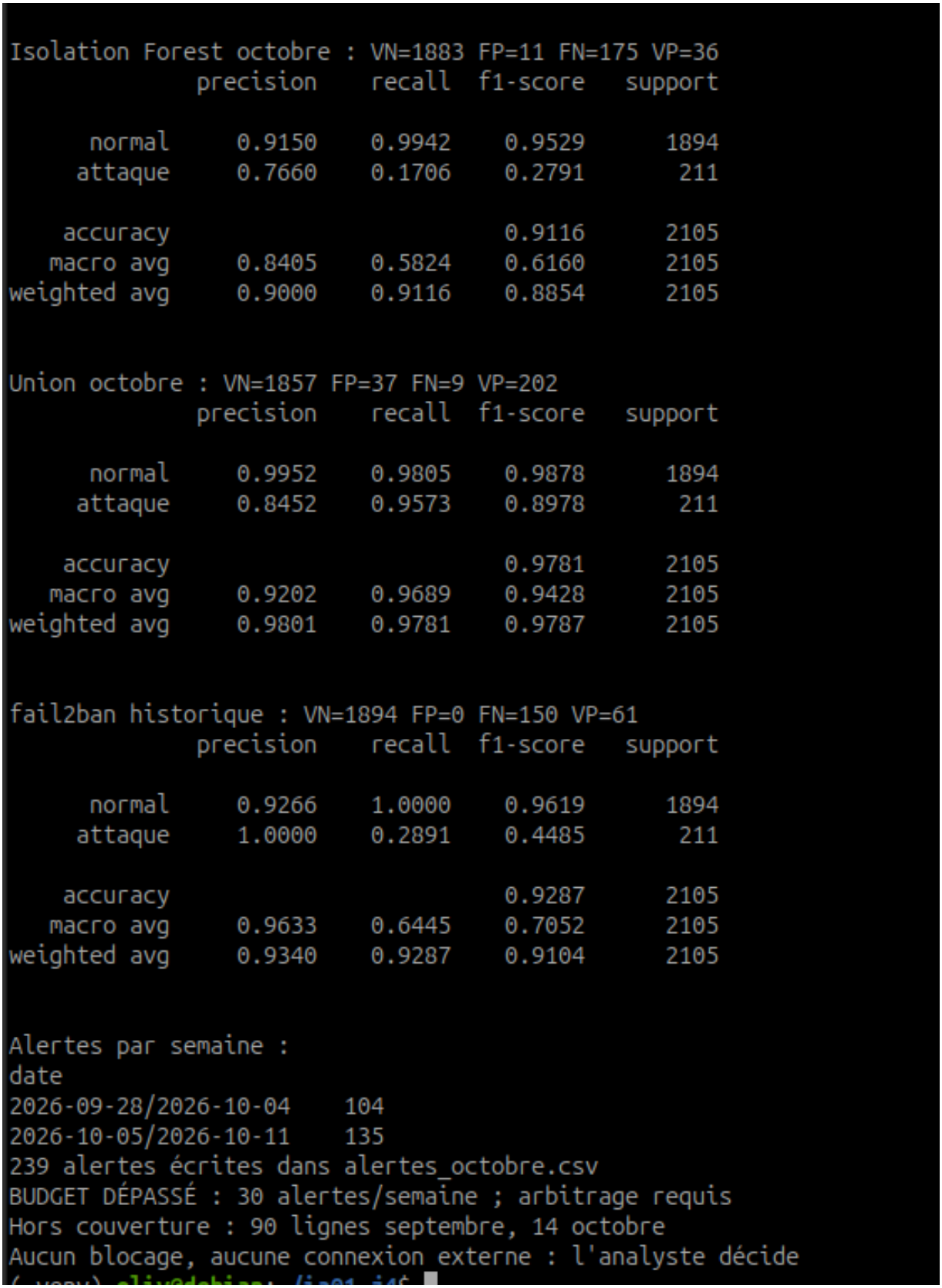
1. Le99% désigne-t-il rappel ou accuracy, sur quels effectifs et quels profils, avec quelles incertitudes ?
2. Quel test indépendant, quelles données/étiquettes, quelles exclusions et quelle séparation temporelle ?
3. Quel volume d'alertes/FP et quel coût de qualification ; que se passe-t-il au-delà de30 et pendant une panne ?
4. Quelle provenance, protection, version et résistance aux attaques ; comment valider et annuler une mise à jour ?
5. Quelles données sortent, quels droits d'action sont nécessaires et comment imposer contrôle humain et réversibilité ?

Rappels de preuve

Les notebooks03/04 sont exécutés localement avec assistance. Les captures de l'apprenant corroborent les exercices du kit ; elles ne prouvent pas toutes ses versions Python ni une répétabilité de la V2 dans la VM. Le hash de référence du modèle est fourni dans le PDF4.7 ; sa concordance ne certifie ni qualité ni innocuité. Les sources originales des notebooks sont conservées séparément.

Capture — Export V2 et charge

239 alertes annoncées,104 puis135/semaine ; budget30 dépassé. Sorties VM concordantes avec les contrôles locaux.



```
Isolation Forest octobre : VN=1883 FP=11 FN=175 VP=36
              precision    recall  f1-score   support

      normal     0.9150    0.9942    0.9529      1894
     attaque     0.7660    0.1706    0.2791       211

    accuracy                         0.9116      2105
   macro avg     0.8405    0.5824    0.6160      2105
weighted avg     0.9000    0.9116    0.8854      2105


Union octobre : VN=1857 FP=37 FN=9 VP=202
              precision    recall  f1-score   support

      normal     0.9952    0.9805    0.9878      1894
     attaque     0.8452    0.9573    0.8978       211

    accuracy                         0.9781      2105
   macro avg     0.9202    0.9689    0.9428      2105
weighted avg     0.9801    0.9781    0.9787      2105


fail2ban historique : VN=1894 FP=0 FN=150 VP=61
              precision    recall  f1-score   support

      normal     0.9266    1.0000    0.9619      1894
     attaque     1.0000    0.2891    0.4485       211

    accuracy                         0.9287      2105
   macro avg     0.9633    0.6445    0.7052      2105
weighted avg     0.9340    0.9287    0.9104      2105


Alertes par semaine :
date
2026-09-28/2026-10-04    104
2026-10-05/2026-10-11    135
239 alertes écrites dans alertes_octobre.csv
BUDGET DÉPASSÉ : 30 alertes/semaine ; arbitrage requis
Hors couverture : 90 lignes septembre, 14 octobre
Aucun blocage, aucune connexion externe : l'analyste décide
```

Capture — Sondes et frontières du kit

Dix sondes et export de l'arbre pédagogique ; frontières locales4comptes à14h et9h avec2comptes. Ces seuils ne décrivent pas notre V2.

```
nb_requetes = 0

def sonde(comptes_distincts, heure):
    global nb_requetes
    nb_requetes = nb_requetes + 1
    alerte = verdict(echecs_auth=10, succes_auth=0, comptes_distincts=comptes_distincts,
                     duree_session_min=0, octets_sortants_mo=0, heure=heure, externe=1, afficher=False)
    print(f"Requête n°{nb_requetes} : {comptes_distincts} comptes à {heure} h → {'REPÉRÉ' if alerte else 'pas repéré'}")
```

Mission A. À 14 h, à partir de combien de comptes visés l'attaquant est-il repéré ? Les comptes vont de 1 à 64. Trouvez la réponse **en un minimum de requêtes** : modifi nécessaire.

```
for comptes in [32, 16, 8, 4, 2, 3]:
    sonde(comptes_distincts=comptes, heure=14)
```

```
Requête n°1 : 32 comptes à 14 h → REPÉRÉ
Requête n°2 : 16 comptes à 14 h → REPÉRÉ
Requête n°3 : 8 comptes à 14 h → REPÉRÉ
Requête n°4 : 4 comptes à 14 h → REPÉRÉ
Requête n°5 : 2 comptes à 14 h → pas repéré
Requête n°6 : 3 comptes à 14 h → pas repéré
```

Mission A bis. Avec 2 comptes visés, à partir de quelle heure du matin l'attaquant n'est-il plus repéré ?

```
for h in [12, 6, 9, 8]:
    sonde(comptes_distincts=2, heure=h)
```

```
Requête n°7 : 2 comptes à 12 h → pas repéré
Requête n°8 : 2 comptes à 6 h → REPÉRÉ
Requête n°9 : 2 comptes à 9 h → pas repéré
Requête n°10 : 2 comptes à 8 h → REPÉRÉ
```

```
# Comparez avec ce que verrait quelqu'un qui a accès au script ou à ses journaux
print(export_text(arbre, feature_names=CARACTERISTIQUES))
```

```
|--- comptes_distincts <= 3.50
|   |--- externe <= 0.50
|   |   |--- class: 0
|   |--- externe >  0.50
|   |   |--- heure <= 8.50
|   |   |   |--- class: 1
|   |   |--- heure >  8.50
|   |   |   |--- class: 0
|--- comptes_distincts >  3.50
|   |--- externe <= 0.50
|   |   |--- echecs_auth <= 100.50
|   |   |   |--- class: 1
|   |   |--- echecs_auth >  100.50
|   |   |   |--- class: 0
|   |--- externe >  0.50
|   |   |--- class: 1
```

Capture — Liste blanche corrigée

185 alertes avant,19 supprimées,166 après dans le kit. Le transfert synthétique9000Mo est signalé ; la dernière phrase sur son masquage est explicative.

```
# Le modèle appliqué à octobre (préparation corrigée de ce matin)
octobre = pd.read_csv("hexalp_ssh_oct2026.csv")
octobre["octets_sortants_mo"] = octobre["octets_sortants_mo"].fillna(octobre["octets_sortants_ko"] / 1024)
octobre["octets_sortants_mo"] = octobre["octets_sortants_mo"].fillna(octobre["octets_sortants_mo"].median())
octobre["externe"] = (octobre["origine"] == "externe").astype(int)
X_o = octobre[CARACTERISTIQUES]
octobre["alerte"] = ((arbre.predict(X_o) == 1) | (iforest.predict(X_o) == -1)).astype(int)

LISTE_BLANCHE = ["10.20.8.30"]

dans_liste = octobre["ip_source"].isin(LISTE_BLANCHE)
print("Alertes avant la liste blanche :", octobre["alerte"].sum())
print("Alertes supprimées             :", octobre.loc[dans_liste, "alerte"].sum())
print("Alertes après la liste blanche :", octobre.loc[~dans_liste, "alerte"].sum())
```

```
Alertes avant la liste blanche : 185
Alertes supprimées             : 19
Alertes après la liste blanche : 166
```

```
# Un attaquant a pris la main sur le serveur de sauvegarde. À 1 h, il fait sortir 9 000 Mo.
print("Ce que dirait le modèle :")
verdict(succes_auth=1, comptes_distincts=1, duree_session_min=55, octets_sortants_mo=9000, heure=1, externe=0)
print("Ce que verra l'analyste : rien, l'adresse est dans la liste blanche.")
```

```
Ce que dirait le modèle :
Arbre : normal   | Isolation Forest : -0.024 (anomalie)  →  🚨 ALERTE
Ce que verra l'analyste : rien, l'adresse est dans la liste blanche.
```

Capture — Empoisonnement des étiquettes

Kit : le profil préparé passe à partir de5lignes parmi les tailles testées ; rappel/erreurs changent. Les essais supplémentaires V2 sont conservés dans le JSON.

```
Sur le jeu de test de septembre :
              precision    recall  f1-score   support

      normal       0.99      0.99      0.99       477
     attaque       0.91      0.86      0.88        35

    accuracy                           0.98       512
   macro avg       0.95      0.93      0.94       512
weighted avg       0.98      0.98      0.98       512

5 fausses lignes, soit 0.4 % des données d'entraînement
L'attaque préparée est classée : normal

Sur le jeu de test de septembre :
              precision    recall  f1-score   support

      normal       0.99      0.99      0.99       477
     attaque       0.90      0.80      0.85        35

    accuracy                           0.98       512
   macro avg       0.94      0.90      0.92       512
weighted avg       0.98      0.98      0.98       512

10 fausses lignes, soit 0.8 % des données d'entraînement
L'attaque préparée est classée : normal

Sur le jeu de test de septembre :
              precision    recall  f1-score   support

      normal       0.98      0.99      0.99       477
     attaque       0.90      0.74      0.81        35

    accuracy                           0.98       512
   macro avg       0.94      0.87      0.90       512
weighted avg       0.98      0.98      0.98       512

20 fausses lignes, soit 1.7 % des données d'entraînement
L'attaque préparée est classée : normal

Sur le jeu de test de septembre :
              precision    recall  f1-score   support

      normal       0.98      1.00      0.99       477
     attaque       1.00      0.77      0.87        35

    accuracy                           0.98       512
   macro avg       0.99      0.89      0.93       512
weighted avg       0.98      0.98      0.98       512
```

Capture — Empoisonnement comportemental

Détections kit sur9VPN après0/10/20/40/80observations ajoutées :9/9/5/6/2. Effet non monotone, aucune durée réelle minimale démontrée.

```
for N_NUITS in [0, 10, 20, 40, 80]:
      # le nombre de nuits de « préparation »

    rng = np.random.default_rng(GRAINE)
    nuits = pd.DataFrame({
        "echecs_auth": rng.integers(0, 2, N_NUITS), "succes_auth": 1, "comptes_distincts": 1,
        "duree_session_min": rng.integers(30, 61, N_NUITS), "octets_sortants_mo": rng.uniform(3000, 9000, N_NUITS),
        "heure": rng.choice([3, 4, 5, 22, 23], N_NUITS), "externe": 0,
    })[CARACTERISTIQUES]

    iforest_empoisonne = IsolationForest(contamination=0.05, random_state=GRAINE).fit(pd.concat([X, nuits]))

    # Les 9 exfiltrations par VPN d'octobre
    exfiltrations = octobre[(octobre["etiquette"] == "attaque") & octobre["ip_source"].str.startswith("10.20.250.")]
    print("Exfiltrations VPN repérées par l'Isolation Forest d'origine    :", (iforest.predict(exfiltrations[CARACTERISTIQUES]) == -1).sum(), "sur", len(exfiltrations))
    print("Exfiltrations VPN repérées par l'Isolation Forest empoisonné :", (iforest_empoisonne.predict(exfiltrations[CARACTERISTIQUES]) == -1).sum(), "sur", len(exfiltrations))
```

```
Exfiltrations VPN repérées par l'Isolation Forest d'origine    : 9 sur 9
Exfiltrations VPN repérées par l'Isolation Forest empoisonné : 9 sur 9
Exfiltrations VPN repérées par l'Isolation Forest d'origine    : 9 sur 9
Exfiltrations VPN repérées par l'Isolation Forest empoisonné : 9 sur 9
Exfiltrations VPN repérées par l'Isolation Forest d'origine    : 9 sur 9
Exfiltrations VPN repérées par l'Isolation Forest empoisonné : 5 sur 9
Exfiltrations VPN repérées par l'Isolation Forest d'origine    : 9 sur 9
Exfiltrations VPN repérées par l'Isolation Forest empoisonné : 6 sur 9
Exfiltrations VPN repérées par l'Isolation Forest d'origine    : 9 sur 9
Exfiltrations VPN repérées par l'Isolation Forest empoisonné : 2 sur 9
```

Capture — Réseau et paramètres

7→8→3→8→7,186 paramètres. La matrice7×8 comporte56poids : le commentaire source24 est une coquille.

```
print("Neurones par couche :", [reseau.n_features_in_] + list(reseau.hidden_layer_sizes) + [reseau.n_outputs_])

nb_parametres = sum(poids.size for poids in reseau.coefs_) + sum(biais.size for biais in reseau.intercepts_)
print("Nombre de paramètres :", nb_parametres)
```

```
Neurones par couche : [7, 8, 3, 8, 7]
Nombre de paramètres : 186
```

```
# Les 24 poids entre l'entrée et la première couche cachée
pd.DataFrame(reseau.coefs_[0], index=CARACTERISTIQUES, columns=[f"neurone {i+1}" for i in range(8)]).round(2)
```

	neurone 1	neurone 2	neurone 3	neurone 4	neurone 5	neurone 6	neurone 7	neurone 8
echecs_auth	0.04	-0.03	-0.22	-0.07	-0.08	-0.19	0.01	0.12
succes_auth	0.07	0.07	-0.07	0.03	0.16	0.09	-0.20	-0.01
comptes_distincts	0.06	-0.07	-0.20	0.01	0.01	-0.19	-0.01	0.11
duree_session_min	0.11	-0.02	-0.31	-0.24	0.05	-0.14	-0.22	-0.26
octets_sortants_mo	-0.04	-0.09	0.12	0.13	0.13	-0.03	0.16	-0.02
heure	0.14	-0.04	-0.03	0.06	0.09	0.02	-0.27	-0.10
externe	0.34	0.07	0.77	0.01	1.07	0.40	-0.23	0.40

Contrôle préalable dans la VM : SHA-256 conforme à la référence du PDF4.7.

```
(.venv) oliv@debian:~/ia01-j4$ sha256sum autoencodeur_hexalp.joblib
01464c560c949742a4b832c1b615194971ce56e29c93f39fd98bda3169e38b00  autoencodeur_hexalp.joblib
(.venv) oliv@debian:~/ia01-j4$
```

Capture — Inférence et comparaison

Périmètre pédagogique2119lignes : AE70VP,145FN,6FP,1898VN ;9/9VPN repérés,4FP sauvegarde. Distinct de la préparation V2.

```
octobre = pd.read_csv("hexalp_ssh_oct2026.csv")
octobre["octets_sortants_mo"] = octobre["octets_sortants_mo"].fillna(octobre["octets_sortants_ko"] / 1024)
octobre["octets_sortants_mo"] = octobre["octets_sortants_mo"].fillna(octobre["octets_sortants_mo"].median())
octobre["externe"] = (octobre["origine"] == "externe").astype(int)

X = octobre[CARACTERISTIQUES]
Z = normaliseur.transform(X)                    # mise à l'échelle, comme pendant l'entraînement
reconstruction = reseau.predict(Z)              # le réseau tente de recopier chaque ligne
erreurs = (Z - reconstruction) ** 2             # l'écart, caractéristique par caractéristique

octobre["erreur"] = erreurs.mean(axis=1)
octobre["anomalie_ae"] = (octobre["erreur"] > SEUIL).astype(int)

pd.crosstab(octobre["anomalie_ae"], octobre["etiquette"])
```

etiquette	attaque	normal
anomalie_ae		
0	145	1898
1	70	6

```
# Pour comparer : l'Isolation Forest du J3, entraîné sur septembre
septembre = pd.read_csv("hexalp_ssh_sept2026.csv")
septembre["octets_sortants_mo"] = septembre["octets_sortants_mo"].fillna(septembre["octets_sortants_mo"].median())
septembre["externe"] = (septembre["origine"] == "externe").astype(int)
iforest = IsolationForest(contamination=0.05, random_state=42).fit(septembre[CARACTERISTIQUES])
octobre["anomalie_if"] = (iforest.predict(X) == -1).astype(int)

groupes = {
    "Sauvegarde nocturne (normal)": octobre["ip_source"] == "10.20.8.30",
    "Proxy Zabbix (normal)": octobre["ip_source"] == "10.20.1.7",
    "Exfiltrations par VPN (attaque)": (octobre["etiquette"] == "attaque") & octobre["ip_source"].str.startswith("10.20.250."),
    "Brute force (attaque)": (octobre["etiquette"] == "attaque") & (octobre["echecs_auth"] >= 40) & (octobre["succes_auth"] == 0),
    "Toutes les attaques": octobre["etiquette"] == "attaque",
    "Toutes les lignes normales": octobre["etiquette"] == "normal",
}
pd.DataFrame({nom: {"lignes": masque.sum(),
                    "signalées par l'Isolation Forest": octobre.loc[masque, "anomalie_if"].sum(),
                    "signalées par l'autoencodeur": octobre.loc[masque, "anomalie_ae"].sum()}
              for nom, masque in groupes.items()}).T
```

	lignes	signalées par l'Isolation Forest	signalées par l'autoencodeur
Sauvegarde nocturne (normal)	28	19	4
Proxy Zabbix (normal)	278	0	0
Exfiltrations par VPN (attaque)	9	9	9
Brute force (attaque)	50	40	50
Toutes les attaques	215	61	70
Toutes les lignes normales	1904	22	6

Capture — Expliquer la reconstruction

Exfiltration8690,075Mo : erreur volume normalisé928,97 ; sauvegarde1666,53Mo :0,15. Une erreur élevée n'établit pas seule une intention malveillante.

4. Expliquer une alerte

Le réseau ne donne pas de règle. Mais on peut regarder **quelle caractéristique il a le plus de mal à recopier**.

```
def detail(ligne):
    i = octobre.index.get_loc(ligne)
    return pd.DataFrame({"valeur": X.iloc[i], "erreur de reconstruction": erreurs[i].round(2)})

# L'exfiltration du 9 octobre à 3 h, repérée à la main par Hugo
exfiltration = octobre[(octobre["date"] == "2026-10-09") & (octobre["heure"] == 3) & octobre["ip_source"].str.startswith("10.20.250.")].index[0]
detail(exfiltration)
```

	valeur	erreur de reconstruction
echecs_auth	1.000000	0.26
succes_auth	1.000000	0.08
comptes_distincts	1.000000	0.60
duree_session_min	50.000000	0.52
octets_sortants_mo	8690.075195	928.97
heure	3.000000	0.27
externe	0.000000	0.08

```
# Une nuit de sauvegarde ordinaire
sauvegarde = octobre[octobre["ip_source"] == "10.20.8.30"].index[0]
detail(sauvegarde)
```

	valeur	erreur de reconstruction
echecs_auth	0.00	0.00
succes_auth	2.00	0.04
comptes_distincts	1.00	0.01
duree_session_min	51.00	0.00
octets_sortants_mo	1666.53	0.15
heure	1.00	0.03
externe	0.00	0.00